programming language evaluator that reads lines of code, processes them using (stack, memory table) => output

LABORATORY 3 · WEEK 3 · 12–16 OCTOBER 2026

Assignment A2: A Toy-Language Interpreter

Object-oriented design (MVC) · generic ADTs · exceptions · **Gradle + Git + JUnit from day one**

#	This week
1 ✓	**Submit assignment A1.**
2	**Set up the project** for the interpreter with Gradle and Git (section 1). You will use the same repository for all the following assignments.
3	**Start A2 in class** and complete it at home. Write JUnit tests as you go (section 5).
4	Check `LabAssignmentsSchedule.pdf` regularly for the assignment schedule.

> **Deadline for A2: week 5 (26–30 October 2026)**
>
> The project is **presented orally to the lab instructor during the laboratory**. At the presentation you pull your tagged repository, run `./gradlew build` (compilation + all tests), run the three example programs and explain your design and code. A submission without passing tests is incomplete.

Overview

Starting with this laboratory you will implement, step by step, an **interpreter for a small toy language**. Each new assignment extends the language (new statements, types, a heap, files, threads …), so the code you write now is the foundation for the whole semester.

The interpreter must use the **Model–View–Controller** architectural pattern and the object-oriented concepts from the lectures: interfaces, polymorphism, generics, exceptions – and, where they fit, modern Java constructs such as records, enums and pattern matching.

From A2 onwards every assignment is delivered as a **Gradle project in a Git repository**, and every assignment comes with **JUnit tests**.

1. Project setup: Gradle, Git and JUnit

Do this **once**, at the beginning of A2. All later assignments continue in the same project and the same repository.

1.1 Tools

Tool	Version	Notes
JDK	25 (current LTS)	e.g. Eclipse Temurin or Oracle JDK; check with `java -version`
Gradle	9.1 or newer	needed only once, to generate the project; afterwards use the wrapper `./gradlew`
Git	any recent version	configure `user.name` and `user.email` before the first commit
IDE	IntelliJ IDEA (recommended)	open the project folder; the IDE imports the Gradle build automatically

1.2 Generating the project

Run the following commands in a terminal (on Windows use `gradlew.bat` instead of `./gradlew`):

terminal

```
$ mkdir toy-interpreter
$ cd toy-interpreter
$ git init
$ gradle init --type java-application --dsl kotlin --test-framework junit-jupiter \
      --package interpreter --project-name toy-interpreter \
      --java-version 25 --no-split-project --use-defaults
$ ./gradlew run      # compiles and runs the generated App.java
$ ./gradlew test     # compiles and runs the tests
```

`gradle init` creates the build files, the **Gradle wrapper** (`gradlew`, `gradlew.bat`, `gradle/wrapper/`) and a `.gitignore`. Everybody who clones the repository builds with exactly the same Gradle version by running `./gradlew` – no local installation needed.

1.3 Project structure

Organise the source code in packages that follow the MVC architecture. Tests mirror the same packages under `src/test/java`:

layout

```
toy-interpreter/
├── settings.gradle.kts
├── gradle/libs.versions.toml           # dependency versions (JUnit)
├── gradlew, gradlew.bat, gradle/wrapper/
├── .gitignore
└── app/
    ├── build.gradle.kts
    └── src/
        ├── main/java/interpreter/
        │   ├── Main.java                # entry point (view start-up)
        │   ├── model/
        │   │   ├── PrgState.java
        │   │   ├── adt/                 # IStack, MyStack, IDictionary, …
        │   │   ├── exp/                 # IExp, ValueExp, VarExp, ArithExp, LogicExp
        │   │   ├── stmt/                # IStmt, CompStmt, AssignStmt, …
        │   │   ├── type/                # IType, IntType, BoolType
        │   │   └── value/               # IValue, IntValue, BoolValue
        │   ├── repository/              # IRepository, Repository
        │   ├── controller/              # Controller
        │   ├── view/                    # text menu, example programs
        │   └── exceptions/              # MyException and subclasses
        └── test/java/interpreter/       # same packages, *Test.java classes
```

1.4 The build file

Edit `app/build.gradle.kts`: remove the sample `guava` dependency, set the main class and forward the keyboard to the program (otherwise the text menu cannot read input under `./gradlew run`):

app/build.gradle.kts

```
plugins {
    application
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(libs.junit.jupiter)
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}
```

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(25)
    }
}

application {
    mainClass = "interpreter.Main"
}

tasks.named<JavaExec>("run") {
    standardInput = System.`in`          // keyboard input for the text menu
}

tasks.named<Test>("test") {
    useJUnitPlatform()
}
```

The JUnit version is written by `gradle init` in `gradle/libs.versions.toml`; you may update it to the latest JUnit release. Delete the generated `App.java` and `AppTest.java` once you have your own `Main` and tests.

1.5 Working with Git

Add IDE files to the generated `.gitignore`, then make the first commit:

`.gitignore`

```
# append to .gitignore
.idea/
*.iml
out/
.vscode/
.DS_Store
```

`terminal`

```
$ git add .
$ git commit -m "Initial Gradle project for the toy-language interpreter"
$ git remote add origin <URL of your repository>
$ git push -u origin main
```

Rules for the whole semester:

- **Commit early and often** – after each small step that compiles and passes the tests. The history must show how the work progressed; a single "final" commit is not acceptable.
- Write **meaningful commit messages** in the imperative: *Add VarDeclStmt and its tests*, *Fix type check in AssignStmt* – not *update* or *asdf*.
- **Never commit** generated files: `build/`, `.gradle/`, IDE folders. The wrapper files (`gradlew`, `gradle/wrapper/`) **must** be committed.
- Mark each submission with a tag: `git tag A2` followed by `git push --tags`. The instructor evaluates the tagged version.

2. The toy language

2.1 Syntax

A program is a statement. The grammar below uses `::=` for "is defined as" and `|` for alternatives:

```
Prg   ::= Stmt

Stmt  ::= Stmt1 ; Stmt2                          (CompStmt)
        | Type Id                                (VarDeclStmt)
        | Id = Exp                               (AssignStmt)
        | print(Exp)                             (PrintStmt)
        | if Exp then Stmt1 else Stmt2           (IfStmt)
        | nop                                    (NopStmt)

Type  ::= int | bool
Value ::= Number                                 (IntValue)
        | true | false                           (BoolValue)

Exp   ::= Value                                  (ValueExp)
        | Id                                     (VarExp)
        | Exp1 + Exp2 | Exp1 - Exp2              (ArithExp)
        | Exp1 * Exp2 | Exp1 / Exp2
        | Exp1 and Exp2 | Exp1 or Exp2           (LogicExp)
```

There is no parser yet: programs are written directly as trees of Java objects (see section 4.9), so operator precedence is given by the shape of the tree.

2.2 Example programs

	Program	Output
Ex1	`int v; v = 2; print(v)`	2
Ex2	`int a; int b; a = 2 + 3 * 5; b = a + 1; print(b)`	18
Ex3	`bool a; int v; a = true; (if a then v = 2 else v = 3); print(v)`	2

3. Execution semantics

3.1 Program state

The interpreter keeps three structures that together form the **program state** (`PrgState`). Each executed program has its own `PrgState`.

Structure	Content	Notation
ExeStack	the statements that still have to be executed; the top is executed next	`{s1 | s2 | s3}` – `s1` on top, `s3` at the bottom
SymTable	the declared variables and their current values	`{v -> 2, a -> false}`
Out	the values printed so far, in order	`{1, 2}` – 1 was printed first

- **Initially:** ExeStack contains the whole program; SymTable and Out are empty.
- **One step:** pop the top statement of ExeStack and apply exactly one rule S1–S6 below.
- **Complete execution:** repeat steps until ExeStack is empty (rule S7). Then SymTable holds all the variables and Out all the printed values.

3.2 Statement rules

Each rule shows what happens when the given statement is on top of the ExeStack. Parts of the state that are not mentioned remain unchanged.

Rule	Top of ExeStack	Effect	Errors
S1	`Stmt1; Stmt2`	push `Stmt2`, then `Stmt1`: `{Stmt1 | Stmt2 | …}`	–
S2	`Id = Exp`	`v = Eval(Exp)`; update `Id -> v` in SymTable	`Id` not declared; type of `v` differs from the declared type of `Id`
S3	`Type Id`	add `Id -> default(Type)` to SymTable, where `default(int) = 0`, `default(bool) = false`	`Id` already declared
S4	`print(Exp)`	append `Eval(Exp)` to Out	errors of `Eval`
S5	`if Exp then S1 else S2`	`c = Eval(Exp)`; push `S1` if `c` is true, otherwise `S2`	`c` is not a boolean
S6	`nop`	nothing	–
S7	*(ExeStack empty)*	execution ends	a further step is an error: "execution stack is empty"

3.3 A complete execution: Example 1

Step	Rule	ExeStack	SymTable	Out
0	–	`{int v; (v = 2; print(v))}`	`{}`	`{}`
1	S1	`{int v | v = 2; print(v)}`	`{}`	`{}`
2	S3	`{v = 2; print(v)}`	`{v -> 0}`	`{}`
3	S1	`{v = 2 | print(v)}`	`{v -> 0}`	`{}`
4	S2	`{print(v)}`	`{v -> 2}`	`{}`
5	S4	`{}`	`{v -> 2}`	`{2}`

Your interpreter must be able to display exactly this sequence of states (section 4.8).

3.4 Expression rules

Expressions are evaluated recursively against the SymTable; they **never change** the program state. Operand types are checked during evaluation.

Rule	Expression	Value	Errors
E1	value: `Number`, `true`, `false`	the constant itself; type `int` or `bool`	–
E2	variable `Id`	`LookUp(SymTable, Id)`; e.g. `LookUp({a -> 2, b -> 3}, a) = 2`	"variable `Id` is not defined"
E3	`Exp1 op Exp2`, op ∈ {+, -, *, /}	evaluate `Exp1`, then `Exp2`; both must be `int`; result `int`	operand 1 / operand 2 is not an integer; division by zero
E4	`Exp1 and Exp2`, `Exp1 or Exp2`	evaluate `Exp1`, then `Exp2`; both must be `bool`; result `bool`	operand 1 / operand 2 is not a boolean

4. Implementation requirements

Implement the interpreter in Java using the **Model–View–Controller** pattern. The code fragments below show the expected design; complete the missing classes yourself. They use records, enums and pattern matching (Lecture 2); the classic style with classes, getters and `instanceof` + cast is also accepted.

Layer	Package	Responsibility
Model	`interpreter.model…`	types, values, expressions, statements, ADTs, `PrgState`
Repository	`interpreter.repository`	stores the program state(s)
Controller	`interpreter.controller`	executes programs: one step / all steps
View	`interpreter.view`	text menu, the hard-coded example programs
Exceptions	`interpreter.exceptions`	the exception hierarchy of the interpreter

> **Naming convention**
>
> **Every interface name starts with a capital I**: `IStmt`, `IExp`, `IType`, `IValue`, `IStack`, `IDictionary`, `IList`, `IRepository`. Classes implementing them have descriptive names without the prefix: `CompStmt`, `ArithExp`, `IntType`, `IntValue`, `MyStack`, `MyDictionary`, `MyList`, `Repository`.

> **Design for extension**
>
> Later assignments add new statements, types, values and expressions. Keep `IStmt`, `IExp`, `IType` and `IValue` as open interfaces so that new classes can be added without changing existing code.

4.1 Types and values

An `IType` knows its default value (rule S3). An `IValue` knows its type. Records give `equals`, `hashCode` and `toString` for free, so `val.getType().equals(new IntType())` works without any extra code.

`model/type, model/value`

```
public interface IType {
    IValue defaultValue();
}

public record IntType() implements IType {
    @Override public IValue defaultValue() { return new IntValue(0); }
    @Override public String toString() { return "int"; }
}

public interface IValue {
    IType getType();
}

public record IntValue(int val) implements IValue {
    @Override public IType getType() { return new IntType(); }
    @Override public String toString() { return Integer.toString(val); }
}

// TODO: BoolType (default value false) and BoolValue
```

4.2 Expressions

Each expression implements `eval`, which receives the SymTable and returns an `IValue`. Represent operators with an **enum** instead of `int` codes or characters.

model/exp

```
public interface IExp {
    IValue eval(IDictionary<String, IValue> symTable) throws MyException;
}

public enum ArithOp {
    ADD("+"), SUB("-"), MUL("*"), DIV("/");
    private final String symbol;
    ArithOp(String symbol) { this.symbol = symbol; }
    @Override public String toString() { return symbol; }
}

public record ArithExp(ArithOp op, IExp left, IExp right) implements IExp {
    @Override
    public IValue eval(IDictionary<String, IValue> symTable) throws MyException {
        if (!(left.eval(symTable) instanceof IntValue(int n1)))
            throw new ExpressionException("first operand is not an integer");
        if (!(right.eval(symTable) instanceof IntValue(int n2)))
            throw new ExpressionException("second operand is not an integer");
        return switch (op) {
            case ADD -> new IntValue(n1 + n2);
            case SUB -> new IntValue(n1 - n2);
            case MUL -> new IntValue(n1 * n2);
            case DIV -> {
                if (n2 == 0) throw new ExpressionException("division by zero");
                yield new IntValue(n1 / n2);
            }
        };
    }
    @Override
    public String toString() { return "(" + left + " " + op + " " + right + ")"; }
}

// TODO: ValueExp (returns its value), VarExp (looks the variable up),
//       LogicOp + LogicExp (and, or – both operands must be booleans)
```

4.3 Statements

Each statement implements `execute`, which applies its rule to the program state and returns the state (later, when the language gets threads, the returned state will be used).

model/stmt

```
public interface IStmt {
    PrgState execute(PrgState state) throws MyException;
}

public record CompStmt(IStmt first, IStmt second) implements IStmt {     // S1
    @Override
    public PrgState execute(PrgState state) {
        state.getExeStack().push(second);
        state.getExeStack().push(first);
        return state;
    }
    @Override public String toString() { return "(" + first + "; " + second + ")"; }
}

public record AssignStmt(String id, IExp exp) implements IStmt {          // S2
    @Override
    public PrgState execute(PrgState state) throws MyException {
        var symTable = state.getSymTable();
        if (!symTable.isDefined(id))
            throw new StatementException("variable " + id + " was not declared");
        IValue val = exp.eval(symTable);
        if (!val.getType().equals(symTable.lookup(id).getType()))
            throw new StatementException("declared type of variable " + id
                    + " and type of the assigned expression do not match");
        symTable.update(id, val);
        return state;
```

```
    }
    @Override public String toString() { return id + " = " + exp; }
}

// TODO: VarDeclStmt(String name, IType type)              – S3
//       PrintStmt(IExp exp)                               – S4
//       IfStmt(IExp cond, IStmt thenS, IStmt elseS)       – S5
//       NopStmt()                                         – S6
```

4.4 Generic ADTs (DSA + OOP + MAP...)

ExeStack is an ADT **Stack**, SymTable an ADT **Dictionary** and Out an ADT **List**. Each ADT consists of a **generic interface** and a **generic class** that implements it by wrapping an appropriate collection from `java.util`. Operations that cannot be performed (e.g. `pop` on an empty stack) throw an `ADTException`.

`model/adt`

placeholder

```
public interface IStack<T> {
    void push(T v);
    T pop() throws ADTException;
    boolean isEmpty();
}

public class MyStack<T> implements IStack<T> {
    private final Deque<T> stack = new ArrayDeque<>();     // java.util

    @Override public void push(T v) { stack.push(v); }

    @Override public T pop() throws ADTException {
        if (stack.isEmpty()) throw new ADTException("pop from an empty stack");
        return stack.pop();
    }

    @Override public boolean isEmpty() { return stack.isEmpty(); }
    @Override public String toString() { return stack.toString(); }
}

// TODO: IDictionary<K, V> / MyDictionary<K, V>: isDefined, lookup, put, update
//       (wraps a HashMap)
//       IList<T> / MyList<T>: add, get, size (wraps an ArrayList)
```

each is a placeholder im itself

4.5 Program state

`PrgState` has the three structures as fields, plus the original program. It provides getters and setters and overrides `toString` to display the whole state.

model/PrgState.java

```
public class PrgState {
    private IStack<IStmt> exeStack;
    private IDictionary<String, IValue> symTable;
    private IList<IValue> out;
    private final IStmt originalProgram;

    public PrgState(IStack<IStmt> exeStack, IDictionary<String, IValue> symTable,
                    IList<IValue> out, IStmt program) {
        this.exeStack = exeStack;
        this.symTable = symTable;
        this.out = out;
        this.originalProgram = program;
        exeStack.push(program);
    }
    // getters, setters, toString ...
}

PrgState state = new PrgState(new MyStack<>(), new MyDictionary<>(),
                              new MyList<>(), program);
```

Note: the original program is kept so that it can be displayed or re-run later. If your statements are immutable (records with final fields), the same object can be stored; if they are mutable classes, store a deep copy.

4.6 Repository

- Interface `IRepository` with, for now, the method `PrgState getCrtPrg()`. It will get more methods in later assignments.
- Class `Repository` implements it and stores the program states in a `List<PrgState>` – later the repository will hold the states of several threads.

4.7 Controller

The controller holds a reference of **interface type** `IRepository`, so the repository implementation can be replaced easily.

controller/Controller.java

```
public PrgState oneStep(PrgState state) throws MyException {
    var stack = state.getExeStack();
    if (stack.isEmpty()) throw new StatementException("program state stack is empty");
    IStmt crtStmt = stack.pop(); not empty → take first statement → execute it
    return crtStmt.execute(state);
}

public void allStep() throws MyException {
    PrgState prg = repo.getCrtPrg();
    display(prg);                          // only if the display flag is on
    while (!prg.getExeStack().isEmpty()) {
        oneStep(prg);                      next
        display(prg);                      next
    }
}
```

- **one-step execution** – executes the statement on top of the ExeStack (one rule S1–S6);
- **complete execution** – executes until the ExeStack is empty;
- **display** – prints the program state after each step when a *display flag* is set (see the trace in section 3.3).

4.8 View

A text menu with two options: **select a program** (from the example programs hard-coded in the code) and **run the selected program** completely. Start it with `./gradlew run`. Errors are caught in the view and shown as messages – the menu must not crash.

4.9 Example programs as Java objects

The example programs from section 2.2 are written directly as object trees, for example in a class `view.Examples`:

`view/Examples.java`

```
// Ex1: int v; v = 2; print(v)
IStmt ex1 = new CompStmt(new VarDeclStmt("v", new IntType()),
        new CompStmt(new AssignStmt("v", new ValueExp(new IntValue(2))),
                new PrintStmt(new VarExp("v"))));

// Ex2: int a; int b; a = 2 + 3 * 5; b = a + 1; print(b)
IStmt ex2 = new CompStmt(new VarDeclStmt("a", new IntType()),
        new CompStmt(new VarDeclStmt("b", new IntType()),
            new CompStmt(new AssignStmt("a",
                    new ArithExp(ArithOp.ADD, new ValueExp(new IntValue(2)),
                        new ArithExp(ArithOp.MUL, new ValueExp(new IntValue(3)),
                                                  new ValueExp(new IntValue(5)))),
                new CompStmt(new AssignStmt("b",
                        new ArithExp(ArithOp.ADD, new VarExp("a"),
                                                  new ValueExp(new IntValue(1)))),
                    new PrintStmt(new VarExp("b"))))));

// Ex3: bool a; int v; a = true; (if a then v = 2 else v = 3); print(v)
IStmt ex3 = new CompStmt(new VarDeclStmt("a", new BoolType()),
        new CompStmt(new VarDeclStmt("v", new IntType()),
            new CompStmt(new AssignStmt("a", new ValueExp(new BoolValue(true))),
                new CompStmt(new IfStmt(new VarExp("a"),
                        new AssignStmt("v", new ValueExp(new IntValue(2))),
                        new AssignStmt("v", new ValueExp(new IntValue(3)))),
                    new PrintStmt(new VarExp("v"))))));
```

4.10 Exceptions

Define your own exception hierarchy, rooted in a checked exception `MyException`. Each error from sections 3.2 and 3.4 must produce an exception with a clear message. At least:

Exception	Thrown when
`MyException`	base class of all interpreter errors (extends `Exception`)
`ADTException`	invalid ADT operation: pop from an empty stack, lookup of a missing key, index out of bounds …
`ExpressionException`	division by zero, operand of the wrong type, undefined variable
`StatementException`	empty execution stack, variable declared twice or not declared, type mismatch in assignment, non-boolean condition

5. Testing with JUnit

Every assignment, starting with A2, must include JUnit tests for the new functionality. Tests are part of the grade.

- Test classes live in `app/src/test/java`, in the **same package** as the tested class, and are named `<Class>Test`.

- Run all tests with `./gradlew test`; `./gradlew build` compiles **and** tests. The HTML report is in `app/build/reports/tests/test/index.html`.
- Every test checks **one behaviour** and has a descriptive name, e.g. `divisionByZeroThrows`.
- Test the **error cases** as well, with `assertThrows`. Each test builds its own fresh state (`@BeforeEach`); tests must not depend on each other or on their order.
- Commit the tests together with the code they test. Before each `git push`, `./gradlew test` must pass.

5.1 Minimum tests for A2

Area	Test at least
ADTs	stack: LIFO order, `pop` on an empty stack throws; dictionary: `put` / `lookup` / `update`, lookup of a missing key throws; list: `add` / `get` / `size`
Expressions	each arithmetic operator; division by zero; non-integer operand; `and` / `or`; non-boolean operand; undefined variable
Statements	each rule S1–S6, including its errors: redeclared variable, assignment to an undeclared variable, type mismatch, non-boolean `if` condition
Controller	complete execution of the three example programs gives the expected Out; `oneStep` on an empty stack throws

5.2 Examples

`src/test/java/interpreter/model/exp/ArithExpTest.java`

```
package interpreter.model.exp;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
// + imports of the interpreter classes

class ArithExpTest {
    private final IDictionary<String, IValue> symTable = new MyDictionary<>();

    private static IExp num(int n) { return new ValueExp(new IntValue(n)); }

    @Test
    void respectsOperatorPrecedenceEncodedInTheTree() throws MyException {
        IExp e = new ArithExp(ArithOp.ADD, num(2),
                              new ArithExp(ArithOp.MUL, num(3), num(5)));
        assertEquals(new IntValue(17), e.eval(symTable));
    }

    @Test
    void divisionByZeroThrows() {
        IExp e = new ArithExp(ArithOp.DIV, num(1), num(0));
        var ex = assertThrows(ExpressionException.class, () -> e.eval(symTable));
        assertEquals("division by zero", ex.getMessage());
    }
}
```

src/test/java/interpreter/controller/ControllerTest.java

```
package interpreter.controller;

class ControllerTest {

    private static PrgState run(IStmt program) throws MyException {
        PrgState state = new PrgState(new MyStack<>(), new MyDictionary<>(),
                                      new MyList<>(), program);
        new Controller(new Repository(state), false).allStep();   // display off
        return state;
    }

    @Test
    void example2PrintsEighteen() throws MyException {
        PrgState state = run(Examples.EX2);
        assertTrue(state.getExeStack().isEmpty());
        assertEquals(new IntValue(18), state.getOut().get(0));
    }
}
```

> **Why tests now?**
>
> The interpreter grows with every assignment. A test suite written now tells you immediately when a later change breaks an older feature (regression), and it documents how each statement and expression is supposed to behave.

6. Submission checklist (A2)

- ☐ The project is a Gradle project generated as in section 1; it builds on a fresh clone with `./gradlew build`.
- ☐ All model classes, the three ADTs, `PrgState`, repository, controller and view are implemented according to section 4.
- ☐ The exception hierarchy covers all the errors of sections 3.2 and 3.4.
- ☐ JUnit tests cover at least the cases of section 5.1, and `./gradlew test` passes.
- ☐ The three example programs run from the menu (`./gradlew run`) and display every intermediate program state.
- ☐ The Git history shows incremental work with meaningful messages; `build/` and IDE files are not committed.
- ☐ The submitted version is tagged `A2` and pushed to the remote repository before the deadline.
- ☐ **The project is presented orally to the lab instructor at the laboratory:** you run the build, the tests and the examples, explain the design (MVC, ADTs, exceptions) and answer questions about any part of your code. Code you cannot explain is not accepted.

Deadline: week 5 (26–30 October 2026).